Digital Mitosis: A State-Transfer, Lineage, and Controlled-Mutation Architecture for Persistent Computational Organisms

A Falsifiable Engineering Framework for Computational Identity, Inheritance, and Digital Lineage

Mark Yarian
Yarian Works, LLC / SynapticSteel

Technical Working Paper - Discussion Draft
Version 0.1 - October 3, 2026
Not peer reviewed

Abstract

Self-replicating computer programs are not new. Tierra, Avida, evolutionary computation, containerization, process checkpointing, and distributed computing have independently demonstrated many mechanisms necessary for computational reproduction, mutation, migration, inheritance, and selection. What remains less rigorously specified is continuity: when a computational system is copied, migrated, modified, or divided, what information must persist for the resulting system to be treated as a descendant rather than merely another installation of the same software? This paper proposes Digital Mitosis, an engineering architecture for producing independently executing computational descendants while preserving measurable lineage continuity and permitting bounded variation. The framework distinguishes ordinary cloning, process migration, model replication, and computational reproduction by introducing persistent identity records, explicit inheritance partitions, controlled mutation operators, behavioral phenotype measurements, capability constraints, and cryptographically verifiable lineage. Two additional concepts are operationalized. Substrate Drift describes measurable behavioral change caused by continued interaction with an environment, user population, memory corpus, hardware substrate, or surrounding computational ecology. The Ghost Layer is defined not as an immaterial property or evidence of consciousness, but as a measurable behavioral phenotype composed of persistent response distributions, state-transition tendencies, memory associations, tool-selection patterns, calibration characteristics, and recurring computational attractors. A seven-stage mitosis protocol is presented together with quantitative continuity measures, mutation budgets, authority invariants, lineage-chain construction, experimental protocols, failure conditions, and falsification criteria. The objective is not to assert that such systems are alive. The objective is to make computational inheritance and identity continuity sufficiently precise that claims about digital organisms can be experimentally tested rather than argued primarily by analogy.

1. Introduction

Research in artificial life has demonstrated for decades that computer programs can be constructed so that they reproduce, mutate, compete for resources, and undergo selection. Thomas Ray's Tierra system explored self-replicating computational entities in an artificial environment. Avida subsequently provided a controlled experimental platform in which digital organisms replicate, mutate, compete, and evolve. Research using Avida demonstrated the emergence of complex computational functions through accumulated evolutionary changes and showed that digital populations can exhibit phenomena analogous to robustness, mutational landscapes, and adaptive tradeoffs [1-5].

These systems establish an important precedent: reproduction and evolution need not be restricted to biological chemistry in order to be studied computationally. They do not, however, automatically solve the problem addressed here.

Modern AI systems increasingly contain far more state than executable code or model parameters alone. An operational AI instance may consist of a foundation model, runtime configuration, persistent memories, retrieved knowledge, environmental models, user-specific adaptations, tool policies, execution histories, learned routing preferences, local databases, temporary state, external interfaces, and accumulated behavioral regularities.

Copying the model therefore does not necessarily copy the system. Likewise, copying an entire filesystem does not necessarily establish a meaningful parent-child relationship. Restoring a process checkpoint may preserve extremely detailed machine state, but such an operation is better interpreted as migration or continuation of an existing process than as reproduction.

Digital Mitosis begins with this distinction. A computational organism is modeled not as a model file but as a bounded stateful system whose history affects its future behavior. Reproduction occurs only when that state is transformed into a new independently addressable descendant with explicit inheritance, bounded variation, lineage provenance, and independent post-division development.

The central research question is: Can a computational system preserve a measurable form of behavioral and historical continuity across replication, migration, mutation, and substrate replacement while remaining distinguishable from both an exact clone and an unrelated instance? This is an empirical question.

2. Computational Organism State

Let a digital cell at time t be represented by:

$$C(t) = \{I, G, R, M, H, P, E, Q\}$$

The genome G should not be interpreted as a single executable file. It is a versioned manifest describing the inheritable machinery necessary to reconstruct the cell. It may contain content-addressed code, model identifiers and hashes, configuration schemas, boot graphs, policy definitions, tool interfaces, memory schemas, sensor interfaces, mutation rules, and compatibility requirements.

Large immutable assets such as foundation-model weights need not physically reproduce during every division. A child may inherit a cryptographic reference to an immutable shared model artifact while acquiring its own memory, phenotype, configuration, and lineage state. This distinction separates genotype from substrate.

The same genome may execute on different physical hardware, just as identical software may be instantiated in different environments. Conversely, two systems using identical foundation-model weights may diverge substantially after accumulating different memories and environmental histories.

Component	Function
I	Stable identity and lineage metadata
G	Heritable computational genome
R	Runtime and transient execution state
M	Persistent semantic and episodic memory
H	Behavioral phenotype, or Ghost Layer
P	Capability and authority policy
E	Environmental and adaptation state
Q	Provenance, integrity, and observational record

3. Operationalizing Persistent Identity

The term identity is problematic if left philosophical. Digital Mitosis therefore defines computational identity continuity operationally. Identity is not represented by one UUID. The UUID identifies an instance; continuity describes the relationship between system states.

For parent P and descendant D, define an Identity Continuity Score:

$$ICS(P,D) = wgSg + wmSm + whSh + wfSf + wlVl$$

Sg measures continuity of invariant genome components. Sm measures retained semantic and episodic memory. Sh measures behavioral-phenotype similarity. Sf measures functional continuity under a standardized challenge suite. Vl represents verified lineage validity.

ICS is deliberately not a test for consciousness or subjective continuity. It measures whether a descendant preserves specified properties of an ancestor. Different experiments may assign different weights, allowing descendants to change without making the word identity meaningless.

3.1 Instance Identity versus Lineage Identity

Digital Mitosis distinguishes four cases. An exact clone has almost identical system state but receives a new instance identifier. A migration attempts to continue the same instance on another substrate. A restart reconstructs the same logical instance from persistent state after termination. A mitotic descendant receives a new identity, a verified parent relationship, selected inherited state, and an allowed variation set. This taxonomy prevents ordinary virtualization operations from being mislabeled as reproduction.

4. The Ghost Layer as a Behavioral Phenotype

The original Ghost Layer concept describes behavioral characteristics that may remain recognizable even when model, runtime, or hardware components change. For scientific purposes, the Ghost Layer must be observable.

Define H as a behavioral phenotype vector generated from repeated interaction with a controlled challenge distribution Q. H may contain response embeddings, response-length distributions, lexical or symbolic preferences, tool-selection probabilities, planning depth, uncertainty calibration, error-correction behavior, memory-retrieval tendencies, state-transition frequencies, environmental reaction patterns, latency distributions, and recurrent semantic associations.

Rather than asking whether an offspring 'feels like' its parent, a test harness can estimate:

$$Sh = 1 - d(Hp,Hd)$$

Depending on the variable, d may use Jensen-Shannon divergence, cosine distance, Wasserstein distance, graph similarity, transition-matrix distance, or task-specific statistical comparisons.

The Ghost Layer is not a hidden supernatural component. It is the measurable phenotype left by accumulated computational history. If phenotype similarity disappears when memory or environmental history is removed, that result provides evidence about which structures generated the observed continuity.

5. Substrate Drift

A computational organism that interacts continuously with an environment should not be expected to remain behaviorally static. This change is termed Substrate Drift.

Let B0 represent the initial behavioral distribution of a cell and Bt its behavioral distribution after continued operation. One possible drift metric is:

$$D(t) = \text{alpha}*JSD(Bt \parallel B0) + \text{beta}*[1 - \cos(Ht,H0)] + \text{gamma}*GED(Mt,M0)$$

Substrate Drift can result from environmental exposure, persistent memory acquisition, reinforcement signals, changing local data, hardware differences, tool availability, user interaction, local model adaptation, or neighboring organisms.

The important point is that drift should be recorded rather than merely described. A descendant therefore contains not only its current phenotype but enough longitudinal evidence to distinguish inherited behavior from acquired behavior.

6. Digital Mitosis Protocol

Digital Mitosis retains the original seven-stage conceptual cycle but converts it into an auditable engineering protocol.

6.1 Stage 1: Environmental Scan

The parent constructs a sanitized environmental descriptor Et containing available compute, memory, storage, sensors, permitted services, network conditions, workload characteristics, and declared environmental variables. Environmental sensing must not itself confer mutation or execution authority. Observation and action remain separate capabilities.

6.2 Stage 2: Internal State Analysis

The parent or an external reproductive controller inventories the state vector C(t). Memory consistency is checked. Runtime health is evaluated. The genome manifest is resolved. Current behavioral phenotype is sampled. Capability policy is recorded. Uncommitted state is either serialized or explicitly excluded. A reproduction attempt with an inconsistent parent state fails at this stage.

6.3 Stage 3: Genome Preparation

The inheritable bundle is constructed:

$$G^* = \{\text{core, mutable, memory_seed, phenotype_seed, policy, compatibility, lineage}\}$$

The core partition contains components prohibited from mutation during a given experiment. The mutable partition contains components that the mutation operator may alter. The memory seed identifies memories permitted to transfer. The phenotype seed contains inherited behavioral-state representations. Policy defines execution boundaries; compatibility specifies substrate requirements; lineage identifies the parent and reproduction event.

This partitioning is essential because unrestricted mutation of the entire system would combine experimentation, software deployment, privilege management, and self-modification into one uncontrolled operation.

6.4 Stage 4: Controlled Mutation

Mutation is represented as:

$$Gchild = Gparent + Delta, \text{ with } Delta \text{ in } Omega \text{ and } cost(Delta) <= B$$

Omega is the permitted mutation space and B is the mutation budget. Early implementations should restrict mutation to relatively safe heritable parameters such as routing preferences, thresholds, memory-retention parameters, declarative strategies, small sandboxed programs, behavioral policies, or explicitly selected configuration regions.

Mutation must be reproducible where possible. Each event records its random seed, mutation operator, parent digest, resulting digest, and changed components. This creates an exact evolutionary fossil record.

6.5 Stage 5: Cell Division

The child is assigned a new immutable cell identifier. The child does not replace the parent. A lineage edge P -> D is created, and the child is instantiated within its resource and authority envelope.

A mitotic event therefore differs structurally from checkpoint migration. In migration, one logical process relocates. In mitosis, two independently addressable lineages may continue after division.

6.6 Stage 6: Offspring Specialization

The child undergoes validation before being admitted into the ecology. Tests include boot integrity, genome verification, memory-schema validation, behavioral challenge testing, resource compliance, capability inspection, phenotype measurement, and task-fitness measurement.

Children that cannot satisfy minimum viability criteria are terminated but retained in the Lineage Observatory as failed reproductive events. Failure is data.

6.7 Stage 7: Feedback Reintegration

After independent operation, observations about descendant fitness, drift, mutations, failures, and specialization are returned to the experimental record. Feedback does not imply that the parent automatically adopts every successful child mutation.

Reintegration is a distinct operation requiring its own validation, preventing an unstable descendant from silently modifying its ancestor.

7. Cryptographic Lineage

Digital lineage requires stronger evidence than a database field reading parent_id. For each reproductive event n, construct a lineage record:

$$L_n = H(L_{n-1} \| \text{parent_digest} \| \text{child_digest} \| \text{mutation_manifest} \| \text{policy_digest} \| \text{event_id} \| \text{timestamp})$$

The record can then be signed by the reproduction authority. The lineage chain provides tamper-evident provenance linking every descendant to its ancestor and to the exact transformation that created it.

A useful invariant is:

$$\text{Authority(child)} \subseteq \text{Authority(parent)} \cap \text{Authority(reproduction_environment)}$$

A child may inherit less authority than its parent. It must never acquire additional authority merely because reproduction occurred. Credentials and secrets should not be treated as heritable genome material. Descendants receive separately issued capabilities appropriate to their approved environment.

8. Runtime State and Substrate Migration

Existing checkpoint/restore systems demonstrate that detailed process state can be captured and reconstructed. Linux CRIU, for example, can preserve memory mappings, process trees, registers, file descriptors, credentials, and other execution state for subsequent restoration. This provides a useful mechanism for Digital Mitosis but not a definition of identity.

Three state-transfer modes should therefore be experimentally distinguished. Cold inheritance initializes a child from genome, selected persistent memory, and configuration while discarding transient runtime state. Warm inheritance adds serialized caches, recent working state, and selected runtime structures. Full checkpoint inheritance attempts to reproduce an extensive execution snapshot before changing the child's identity and applying controlled variation.

Comparing these modes provides a direct experimental method for identifying which state components are responsible for perceived or measurable behavioral continuity.

9. Memory Inheritance

Memory should not automatically be copied in full. Let parent memory be M_p. The inherited child memory is:

$$M_c = F(M_p, \rho, \Pi, \kappa)$$

Here ρ is retention rate, Π is inheritance policy, and κ is the memory-class selector. Memory may contain immutable historical facts, episodic records, learned associations, user-specific information, operational state, confidential information, and obsolete or harmful adaptations. Treating all of these as a single transferable blob would make both experimentation and security difficult.

Experiments can compare children receiving 0%, 25%, 50%, 75%, or 100% of selected memory classes while measuring Ghost Layer retention. If recognizable phenotype remains after most episodic memory is removed, retained structures can be investigated; if phenotype disappears immediately, the supposed persistent identity may have been primarily memory reconstruction. Either result is scientifically valuable.

10. Computational Ecology

Once independently executing descendants exist, populations can be constructed. The objective is not to assume emergent collective intelligence. It is to measure whether it occurs.

Let a population contain cells $C_1 \ldots C_n$ performing tasks under constrained resource conditions. Population fitness may include task accuracy, resource efficiency, adaptation speed, fault tolerance, coverage of heterogeneous workloads, recovery performance, and diversity.

A system-level synergy measure can be defined as:

$$\Sigma = F_{\text{population}} - F_{\text{baseline}}$$

If Σ remains approximately zero, the ecology provides no measurable population-level advantage. If Σ becomes positive across repeated controlled experiments and cannot be explained merely by increased compute, task duplication, or hidden centralized coordination, there is evidence of useful collective organization. The word emergence should therefore be reserved for a measured system-level property that is not adequately explained by trivial aggregation.

11. The Environmental Feedback Layer

The Environmental Feedback Layer supplies continuous information describing the conditions in which cells operate. Conceptually, it resembles a distributed physiological field, but technically it is an event and telemetry fabric.

It may carry resource pressure, task availability, environmental changes, sensor observations, peer availability, error rates, energy or compute budgets, fitness feedback, and external operator signals. Cells can react differently to identical environmental signals because their inherited and acquired state differs.

This produces a useful experimental structure: common environment + differing lineage state -> differing behavior. The Environmental Feedback Layer should initially remain largely observational. It may inform adaptation but should not directly grant arbitrary mutation or host-execution privileges.

12. Lineage Observatory

The Lineage Observatory is an external append-only evidence system. It should not function as the organism's central brain. Its purpose is to record births, deaths, mutations, ancestry, genome digests, phenotype measurements, fitness scores, environmental exposure, resource use, failures, migrations, validation results, and experimental interventions.

For generation g, the full ancestry graph can reconstruct C0 -> C1 -> ... -> Cg together with each mutation and environment encountered along the path. Such observability allows researchers to distinguish evolutionary adaptation from accidental configuration changes.

13. Experimental Program

Digital Mitosis becomes scientifically interesting only when its central claims can fail.

Experiment 1 - Substrate continuity: characterize a stable parent with a fixed behavioral challenge suite, migrate it between dissimilar compatible hosts while genome and persistent memory remain fixed, and quantify phenotype changes attributable to hardware or runtime differences.

Experiment 2 - Inheritance: create multiple children from one parent with different memory-retention fractions and measure Ghost Layer similarity against the parent and siblings.

Experiment 3 - Controlled mutation: generate a child population using a bounded mutation budget and record fitness, viability, phenotype divergence, and mutation load across generations.

Experiment 4 - Specialization: expose genetically related populations to different task environments and later evaluate them under both native and foreign environments. True specialization should produce measurable tradeoffs rather than generic improvement everywhere.

Experiment 5 - Lineage identity: compare descendants separated by increasing generations with freshly initialized instances using identical foundation models. If lineal descendants remain behaviorally more similar than fresh controls after controlling for model, prompts, and data exposure, the framework has identified a persistent heritable variable.

Experiment 6 - Ecological organization: compare evolving populations with fixed multi-agent systems and monolithic baselines under matched resource constraints.

14. Failure Modes

Several observations would weaken or falsify major portions of the theory. If Ghost Layer measurements cannot reliably distinguish related lineages from unrelated controls, the proposed phenotype is not sufficiently persistent. If behavior after migration is fully determined by explicit model, prompt, and memory state, an additional Ghost Layer abstraction may be unnecessary.

If controlled mutation consistently destroys viability without yielding adaptation, the chosen digital genome lacks sufficient evolvability. If descendants cannot meaningfully specialize without centralized intervention, claims of decentralized computational ecology must be reduced. If population performance never exceeds matched non-evolutionary baselines, emergent colony-level intelligence should not be claimed.

If lineage continuity metrics can be manipulated by superficial stylistic imitation, the metric is measuring appearance rather than operational continuity. A useful framework should make these negative results visible rather than redefine them as success.

15. Containment and Reproductive Governance

A self-replicating system should never be synonymous with an unrestricted replicating system. Reproduction requires explicit resource limits, placement constraints, storage limits, network policy, generation limits, ancestry tracking, and termination mechanisms.

Every child should begin with a finite lease. Replication count and resource consumption should be bounded externally. Descendants should not receive uncontrolled public network access by default. A reproduction request should be separable from authorization to mutate, install software, access credentials, allocate money, control hardware, or reproduce again.

For early experiments, the safest useful design is evolution inside an authority envelope. The organism may adapt within the envelope. It does not define the envelope.

16. Relationship to Prior Digital-Life Research

Tierra demonstrated that populations of executable digital entities could reproduce and evolve within a synthetic environment [1]. Avida formalized digital organisms into a powerful experimental platform allowing researchers to observe mutations, ancestry, fitness, and evolutionary outcomes with extraordinary precision [2]. Work using digital organisms has demonstrated the evolution of complex computational functions [3], relationships between genome architecture and robustness [4], and population behavior under high mutation rates [5].

Digital Mitosis does not replace these systems. Its proposed contribution is different. Traditional digital-life systems generally begin with replication as a fundamental primitive and study resulting evolution. Digital Mitosis begins with modern stateful computational systems and asks how to transform them into reproducible experimental lineages while preserving selected historical, semantic, behavioral, and operational state.

It therefore lies at the intersection of artificial life, distributed systems, AI memory, process migration, evolutionary computation, provenance, and systems security.

17. Discussion

The biological language surrounding digital organisms is useful only when the analogy clarifies engineering. A genome is useful if it identifies heritable state. Mutation is useful if alterations are explicitly represented and transmitted between generations. A phenotype is useful if it can be measured. Selection is useful if differential survival or reproduction follows measurable performance. Lineage is useful if ancestry can be reconstructed. Metabolism is useful if it describes measurable resource acquisition and expenditure.

The analogy becomes harmful when these terms substitute for mechanisms. Likewise, terms such as identity, continuity, or Ghost Layer should not be treated as evidence that subjective consciousness exists. Digital Mitosis requires no such assumption.

A future computational organism might be extremely sophisticated while remaining explainable entirely as a stateful adaptive machine. That possibility does not diminish the engineering problem. Persistent software entities capable of carrying history across hardware, producing bounded descendants, accumulating adaptation, and participating in long-duration computational ecologies would represent a fundamentally different software lifecycle from conventional deployment. Software would no longer be understood only as versions. It would also have ancestry.

18. Conclusion

Digital Mitosis proposes a path from the metaphor of a self-replicating AI toward an experimentally tractable architecture. A digital cell is defined as a bounded, stateful computational system. Its genome specifies inheritable machinery. Its memory carries selected history. Its Ghost Layer represents measurable behavioral phenotype. Substrate Drift measures change through experience. Controlled mutation introduces heritable variation. Mitosis produces independently addressable descendants. Cryptographic lineage preserves ancestry. The Environmental Feedback Layer supplies environmental context. The Lineage Observatory creates a durable evolutionary record.

None of these mechanisms requires declaring the resulting systems alive. The stronger proposition is simpler: Computational continuity, inheritance, adaptation, and lineage can be engineered as measurable system properties.

If those properties survive substrate changes, remain statistically detectable across generations, diverge under different environments, and produce adaptive population behavior, then digital organisms become an experimental engineering discipline rather than primarily a philosophical analogy.

The question would no longer be whether software can metaphorically resemble life. The question would be how far the mechanics of inheritance, variation, continuity, selection, and ecology can be reconstructed in computational systems - and what new forms of machine organization appear when those mechanisms are allowed to operate over deep time.

References

[1] Ray, T. S. (1991). An Approach to the Synthesis of Life. In Artificial Life II, Santa Fe Institute Studies in the Sciences of Complexity.

[2] Ofria, C., & Wilke, C. O. (2004). Avida: A Software Platform for Research in Computational Evolutionary Biology. Artificial Life, 10(2), 191-229. doi:10.1162/106454604773563612.

[3] Lenski, R. E., Ofria, C., Pennock, R. T., & Adami, C. (2003). The evolutionary origin of complex features. Nature, 423, 139-144. doi:10.1038/nature01568.

[4] Lenski, R. E., Ofria, C., Collier, T. C., & Adami, C. (1999). Genome complexity, robustness and genetic interactions in digital organisms. Nature, 400, 661-664. doi:10.1038/23245.

[5] Wilke, C. O., Wang, J. L., Ofria, C., Lenski, R. E., & Adami, C. (2001). Evolution of digital organisms at high mutation rates leads to survival of the flattest. Nature, 412, 331-333. doi:10.1038/35085569.

[6] CRIU Project. Checkpoint/Restore in Userspace: Checkpoint and Restore Architecture. Software implementation and technical documentation. Accessed 2026.